A Minimal Calculus of Implicit Implementations

Scala 3 reference model 28 October 2026

1 Types, expressions, and contexts

$$\begin{array}{rcl} A, B, P, Q & ::= & \alpha \mid \mathsf{Int} \mid \mathsf{Bool} \mid \mathsf{String} \mid A \to B \mid A \Rightarrow B \mid \forall\alpha.A \mid \{\ell_j : A_j\}_{j\in J} \\ \Gamma & ::= & \cdot \mid \Gamma, \alpha \mid \Gamma, x : A \\ \Delta & ::= & [\,] \mid (d : A) :: \Delta \end{array}$$

Γ binds type variables and ordinary values; Δ binds available implementation values, nearest first. P, Q are ordinary types, not a distinct predicate sort. Record labels are distinct and canonically ordered. Equality is structural, modulo α-equivalence and lexical type-synonym expansion. There is no subtyping.

Expression notation for the typing rules (with resolved annotation types):

$$\begin{array}{rcl} e & ::= & x \mid d \mid n \mid b \mid s \mid \lambda(x : A).\, e \mid \lambda\{d : A\}.\, e \mid e(e) \mid e\{e\} \\ & & \mid \Lambda\alpha.e \mid e[A] \mid \mathsf{let}\ x = e\ \mathsf{in}\ e \mid \mathsf{impl}\ d : A = e\ \mathsf{in}\ e \\ & & \mid \{\ell_j = e_j\}_{j\in J} \mid e.\ell \mid (e : A) \mid \mathsf{need}[A] \mid e_1 \odot e_2 \end{array}$$

Before elaboration, annotation types may contain unexpanded names $C[\bar{A}]$, and expressions may contain lexical bindings $\mathsf{type}\ C[\bar{a}] = A\ \mathsf{in}\ e$. Section 2 gives the source-to-AST translation; elaboration resolves annotations and expands type synonyms. The syntax $e.\ell$ is an unclassified selection with an arbitrary term receiver. Elaboration determines whether a selection becomes a core projection or a method applied to its receiver. Both x and d are parsed as names; lexical name resolution distinguishes their binders. In rules, bound names are globally fresh by α-renaming. This convention preserves ordinary source shadowing without conflating the two environments.

Unresolved core:

$$\begin{array}{rcl} c & ::= & x \mid d \mid n \mid b \mid s \mid \lambda(x : A).\, c \mid \lambda\{d : A\}.\, c \mid c\,c \mid c\{c\} \\ & & \mid \Lambda\alpha.c \mid c[A] \mid \mathsf{let}\ x : A = c\ \mathsf{in}\ c \mid \mathsf{impl}\ d : A = c\ \mathsf{in}\ c \\ & & \mid \{\ell_j = c_j\}_{j\in J} \mid c.\ell \mid ?_A \mid c_1 \odot c_2 \end{array}$$

The core has no aliases, traits, definitions, modules, or method-resolution operation. $?_A$ is a deferred request, not a runtime term.

Primitive binary operators:

$\odot$	$\arg(\odot)$	$\mathrm{res}(\odot)$
$+, -, \times$	Int	Int
$==, <, >$	Int	Bool
$++$	String	String

$$\begin{array}{lll} \delta_+(m,n) = m+n, & \delta_-(m,n) = m-n, & \delta_\times(m,n) = mn, \\ \delta_{==}(m,n) = (m=n), & \delta_<(m,n) = (m<n), & \delta_>(m,n) = (m>n), \\ \delta_{++}(s,t) = st. & & \end{array}$$

Here $b \in \{\mathsf{true}, \mathsf{false}\}$; comparisons produce Boolean values.

Type formation (contexts are assumed well formed):

$$\frac{\alpha \in \Gamma}{\Gamma \vdash \alpha\ \mathsf{type}}\ \text{W-Var} \qquad \frac{B \in \{\mathsf{Int}, \mathsf{Bool}, \mathsf{String}\}}{\Gamma \vdash B\ \mathsf{type}}\ \text{W-Base}$$

$$\frac{\Gamma \vdash A\ \mathsf{type} \qquad \Gamma \vdash B\ \mathsf{type} \qquad \star \in \{\to, \Rightarrow\}}{\Gamma \vdash A \star B\ \mathsf{type}}\ \text{W-Arrow} \qquad \frac{\Gamma, \alpha \vdash A\ \mathsf{type} \qquad \alpha\ \mathsf{fresh}}{\Gamma \vdash \forall\alpha.A\ \mathsf{type}}\ \text{W-All}$$

$$\frac{\Gamma \vdash A_j\ \mathsf{type} \quad (j \in J) \qquad \ell_j\ \text{distinct}}{\Gamma \vdash \{\ell_j : A_j\}_{j\in J}\ \mathsf{type}}\ \text{W-Record}$$

2 Practical syntax and the first AST

The source language translates into the named abstract syntax used by the calculus. Parsing determines grouping and expands syntactic sugar: parameter lists become nested binders, calls become nested applications, and declarations bind the rest of the expression. Names, type annotations, and selections remain unresolved until elaboration.

Write $\hat{e}$ for the abstract expression obtained from source e, and $\hat{A}$ for the named type annotation obtained from source A. The mappings below use the mathematical constructors of Section 1, extended with lexical type bindings and unexpanded type names.

Types and expression forms

Source type	Named abstract type
`Int`, `String`, `Bool`	Int, String, Bool
C, $C[A_1, \ldots, A_n]$	C, $C[\hat{A}_1, \ldots, \hat{A}_n]$
`A -> B`	$\hat{A} \to \hat{B}$
`A ~> B`	$\hat{A} \Rightarrow \hat{B}$
`forall` $a_1, \ldots, a_n$ `.` B	$\forall a_1. \cdots \forall a_n. \hat{B}$
$\{\ell_1 : A_1, \ldots, \ell_n : A_n\}$	$\{\ell_1 : \hat{A}_1, \ldots, \ell_n : \hat{A}_n\}$
(A)	$\hat{A}$

Both arrows associate to the right. A type name is retained as written; elaboration determines whether it denotes a bound type variable or a lexical synonym.

Names and literals translate to x, n, b, and s; string escapes are decoded. The remaining nonbinding forms translate as follows:

Source expression	Named abstract expression
$\{\ell_1 = e_1, \ldots, \ell_n = e_n\}$	$\{\ell_1 = \hat{e}_1, \ldots, \ell_n = \hat{e}_n\}$
`e: A`	$(\hat{e} : \hat{A})$
`summon[A]`	need$[\hat{A}]$
`e.field`	$\hat{e}.\mathit{field}$
`P::field`	(need$[\hat{P}]$).field
`f(a)`, `f a`	$\hat{f}(\hat{a})$
`f(using a)`	$\hat{f}\{\hat{a}\}$
`f[A]`	$\hat{f}[\hat{A}]$
$e_1 \odot e_2$	$\hat{e}_1 \odot \hat{e}_2$
`(e)`, `{ e }`	$\hat{e}$

Field assignments distinguish a record literal from a brace group; `{}` is the empty record. A grouped expression can itself contain declarations. Consequently, `f { e }` translates to the ordinary application $\hat{f}(\hat{e})$. Binary operators use the primitive operations specified in Section 1.

Binders and application lists

A parameter group contains one or more comma-separated entries `x: A` or `implicit d: P`. Concatenate repeated groups in source order to obtain $\bar{p} = [p_1, \ldots, p_n]$. Each entry wraps an abstract body E:

$$\mathcal{L}_{x:A}(E) = \lambda(x : \hat{A}).\, E,$$
$$\mathcal{L}_{\texttt{implicit } d:P}(E) = \lambda\{d : \hat{P}\}.\, E,$$
$$\mathcal{L}_{\bar{p}}(E) = \mathcal{L}_{p_1}(\mathcal{L}_{p_2}(\cdots \mathcal{L}_{p_n}(E) \cdots)).$$

A lambda `(`$\bar{p}$`) => e` translates to $\mathcal{L}_{\bar{p}}(\hat{e})$. Comma-separated parameters and repeated groups therefore produce the same nested binders, with the first parameter outermost. For example:

$$\texttt{(a: A, implicit b: B, c: C) => e} \quad \longmapsto \quad \lambda(a : \hat{A}).\, \lambda\{b : \hat{B}\}.\, \lambda(c : \hat{C}).\, \hat{e}.$$

Type lambdas `[`$a_1, \ldots, a_n$`] => e` translate to $\Lambda a_1. \cdots \Lambda a_n.\hat{e}$.

Argument lists instead accumulate applications from left to right. Each `using` marks just its own argument; type-argument lists likewise produce successive type applications:

$$\texttt{f(a, using b, c)} \longmapsto ((\hat{f}(\hat{a}))\{\hat{b}\})(\hat{c}),$$
$$\texttt{f[A, B]} \longmapsto (\hat{f}[\hat{A}])[\hat{B}].$$

Automatic insertion of missing implicit arguments belongs to elaboration of ordinary application; the parser preserves the applications actually written.

Declarations and generic headers

A declaration takes a continuation b, the rest of the expression. In the mappings below, `;` stands for `in`, a semicolon, or a newline. A sequence therefore becomes nested bindings around its final expression.

Source declaration and continuation	Named abstract expression
`let x = e; b`	let $x = \hat{e}$ in $\hat{b}$
`let x: A = e; b`	let $x = (\hat{e} : \hat{A})$ in $\hat{b}$
`type` $C[\bar{a}]$ `= A; b`	type $C[\bar{a}] = \hat{A}$ in $\hat{b}$
`trait` $C[\bar{a}]$ `= A; b`	type $C[\bar{a}] = \hat{A}$ in $\hat{b}$

An omitted type-parameter list supplies the empty list in a lexical type binding.

For definitions and implementations, a generic header $H =$ `[`$a_1, \ldots, a_n$ `where` $P_1, \ldots, P_k$`]` has two effects. It wraps the value in type and implicit abstractions, and wraps a declared implementation type in the corresponding quantifiers and arrows:

$$\mathcal{G}_H(E) = \Lambda a_1. \cdots \Lambda a_n.\lambda\{d_1 : \hat{P}_1\}. \cdots \lambda\{d_k : \hat{P}_k\}.\, E,$$
$$\mathcal{Q}_H(T) = \forall a_1. \cdots \forall a_n.\hat{P}_1 \Rightarrow \cdots \Rightarrow \hat{P}_k \Rightarrow T.$$

The d_j are fresh binders with no source-accessible names. The type-parameter list may be empty, and the `where` clause is optional. An omitted header makes both constructions the identity. Constraints are separated by `and` or commas. Using the parameter construction $\mathcal{L}$ above, a definition translates as

$$\texttt{def } f\, H\, \texttt{(}\bar{p}\texttt{): R = e; b}$$
$$\longmapsto \text{let } f = \mathcal{G}_H(\mathcal{L}_{\bar{p}}((\hat{e} : \hat{R}))) \text{ in } \hat{b}.$$

The result annotation sits inside all the function's binders. The `where` constraints precede the explicitly written ordinary and implicit parameters.

Implementations have the following forms:

```
impl Q = e; b
impl name: Q = e; b
impl [A where P]: Q = e; b
impl name[A where P]: Q = e; b
```

All four translate to

$$\text{impl } d : \mathcal{Q}_H(\hat{Q}) = \mathcal{G}_H(\hat{e}) \text{ in } \hat{b},$$

where d is the supplied name or a fresh source-inaccessible binder, and an absent H is empty. The colon separates a name or header from the target type, so `impl C[A] = e` has target type `C[A]`. Every form requires `=`, followed by any expression.

For example, the definition

```
def display[A where ToString[A]](x: A): String = x.toString
rest
```

translates to

$$\mathsf{let\ display} = \Lambda A.\lambda\{d : \mathsf{ToString}[A]\}.\,\lambda(x : A).\,(x.\mathsf{toString} : \mathsf{String})\ \mathsf{in\ rest}.$$

Here d is the generated constraint binder. The type name $\mathsf{ToString}[A]$, selection $x.\mathsf{toString}$, and result ascription remain in the abstract expression for elaboration.

Grouping and the elaboration boundary

Precedence, from tightest to loosest, is adjacent postfix operations, application, `*`, `+`/`-`, `++`, comparisons, and ascription. Applications and binary operators associate to the left. Adjacency determines the argument tree: `f g(x)` parses as `f(g(x))`, while `f g (x)` parses as `f(g)(x)`. A negative argument is parenthesized, as in `f(-1)`; `f - 1` is subtraction.

A newline after a complete expression ends its application, selection, ascription, and operator chain, allowing it to separate a declaration initializer from its continuation. Line breaks are allowed where a component is still required, such as after `=` or `=>`. Record fields require commas, semicolons, or newlines, with an optional trailing separator. Comments are whitespace; newlines inside them also act as boundaries.

The resulting expression records syntax and lexical nesting. Elaboration resolves term names against ordinary and implementation binders, resolves type names against type variables and synonyms, and checks annotations. It classifies each selection $e.\ell$ using the receiver's type: an existing record field becomes a projection; otherwise the method rule may introduce an implementation request and apply its selected member to the receiver. For `P::field`, parsing has already made that request explicit as $(\mathsf{need}[\hat{P}]).\mathit{field}$. Types remain separate from term receivers: `ToString[A].toString` fails elaboration when `ToString` names only a type. Evidence resolution later supplies implementations for the resulting requests.

Worked translations

The following complete programs illustrate how the translations compose. Symbols such as v and e_{body} abbreviate subexpressions of the displayed mathematical tree. Generated binders are fresh and source-inaccessible.

1. Generic definitions with mixed parameters and arguments.

```
def choose[A where A](fallback: A, implicit preferred: A)(last: A): A = preferred
impl Int = 11
choose[Int](0, using 21, 99)
```

The definition introduces its type parameter, its generated constraint parameter, and then its three written parameters, in that order:

$$v_{\mathsf{choose}} = \Lambda A.\lambda\{d : A\}.\,\lambda(\mathsf{fallback} : A).\,\lambda\{\mathsf{preferred} : A\}.\,\lambda(\mathsf{last} : A).\,(\mathsf{preferred} : A),$$
$$e = \mathsf{let\ choose} = v_{\mathsf{choose}}\ \mathsf{in\ impl}\ i : \mathsf{Int} = 11\ \mathsf{in}\ ((\mathsf{choose}[\mathsf{Int}](0))\{21\})(99).$$

Only the argument 21 is marked implicit in the parsed call. When elaboration reaches the first ordinary argument 0, it inserts the request for the leading `where` constraint. The unresolved call is therefore

$$((\mathsf{choose}[\mathsf{Int}]\{?_{\mathsf{Int}}\})(0))\{21\}(99).$$

Resolution supplies 11 for that request; 21 is supplied directly to preferred. The program returns 21. Replacing the two parameter groups by `(fallback: A)(implicit preferred: A)(last: A)` gives the same abstract definition.

2. Typed bindings, brace groups, and local implementations.

```
type Box[A] = { value: A }
let box: Box[Int] = {
  let seed: Int = 20
  impl Int = seed + 1
  { value = summon[Int] }
}
box.value
```

The initializer's outer braces group its nested bindings; the inner field assignment constructs a record. Both type annotations become ascriptions:

$$e_{\text{body}} = \mathsf{let\ seed} = (20 : \mathsf{Int})\ \mathsf{in\ impl}\ d : \mathsf{Int} = \mathsf{seed} + 1\ \mathsf{in}\ \{\mathsf{value} = \mathsf{need}[\mathsf{Int}]\},$$
$$e = \mathsf{type\ Box}[A] = \{\mathsf{value} : A\}\ \mathsf{in}$$
$$\mathsf{let\ box} = (e_{\text{body}} : \mathsf{Box}[\mathsf{Int}])\ \mathsf{in\ box.value}.$$

The newline after the closing brace starts the continuation of `let box`. The implementation is available while constructing the record; its scope ends before box.value. The record retains the value 21, which is the program's result.

3 Typing and elaboration: functions and bindings

Judgments and name resolution

$$\boxed{\Gamma;\Delta \vdash e : A \rightsquigarrow c} \qquad \boxed{\Gamma;\Delta \vdash c \Longrightarrow t} \qquad \boxed{H \vdash t : T} \qquad \boxed{\rho \vdash t \Downarrow v}$$

These are, respectively, name/type elaboration, evidence resolution, explicit-target typing, and evaluation. The implementation uses separate de Bruijn indices for Γ and Δ, and a type-variable depth. The final target has one term-index space.

A nonrecursive synonym $C[\bar{\alpha}] = B$ expands $C[\bar{A}]$ to $B[\bar{A}/\bar{\alpha}]$ in its lexical body. Expansion is capture avoiding: B retains its declaration-time type-variable bindings. $|\bar{A}| = |\bar{\alpha}|$ is required. Types and values are resolved in their respective namespaces.

All annotation types must satisfy $\Gamma \vdash A\ \mathsf{type}$. Instance selection does not occur in this judgment.

$$\frac{x : A \in \Gamma}{\Gamma;\Delta \vdash x : A \rightsquigarrow x}\ \text{E-Var} \qquad \frac{d : A \in \Delta}{\Gamma;\Delta \vdash d : A \rightsquigarrow d}\ \text{E-Instance}$$

$$\frac{\Gamma, x : A;\Delta \vdash e : B \rightsquigarrow c}{\Gamma;\Delta \vdash \lambda(x : A).\, e : A \to B \rightsquigarrow \lambda(x : A).\, c}\ \text{E-Lambda}$$

$$\frac{\Gamma;(d : A) :: \Delta \vdash e : B \rightsquigarrow c}{\Gamma;\Delta \vdash \lambda\{d : A\}.\, e : A \Rightarrow B \rightsquigarrow \lambda\{d : A\}.\, c}\ \text{E-Implicit-Lambda}$$

$$\frac{\Gamma, \alpha;\Delta \vdash e : A \rightsquigarrow c \qquad \alpha \notin \mathrm{FV}(\Gamma, \Delta)}{\Gamma;\Delta \vdash \Lambda\alpha.e : \forall\alpha.A \rightsquigarrow \Lambda\alpha.c}\ \text{E-Type-Lambda}$$

$$\frac{\Gamma;\Delta \vdash e : \forall\alpha.A \rightsquigarrow c \qquad \Gamma \vdash B\ \mathsf{type}}{\Gamma;\Delta \vdash e[B] : A[B/\alpha] \rightsquigarrow c[B]}\ \text{E-Type-App}$$

$$\frac{\Gamma;\Delta \vdash e_1 : A \Rightarrow B \rightsquigarrow c_1 \qquad \Gamma;\Delta \vdash e_2 : A \rightsquigarrow c_2}{\Gamma;\Delta \vdash e_1\{e_2\} : B \rightsquigarrow c_1\{c_2\}}\ \text{E-Implicit-App}$$

$$\frac{\Gamma;\Delta \vdash e_1 : T \rightsquigarrow c_1 \qquad c_1 : T \triangleright c_1' : A \to B \qquad \Gamma;\Delta \vdash e_2 : A \rightsquigarrow c_2}{\Gamma;\Delta \vdash e_1(e_2) : B \rightsquigarrow c_1'\, c_2}\ \text{E-App}$$

$$\frac{\Gamma;\Delta \vdash e_1 : A \rightsquigarrow c_1 \qquad \Gamma, x : A;\Delta \vdash e_2 : B \rightsquigarrow c_2}{\Gamma;\Delta \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 : B \rightsquigarrow \mathsf{let}\ x : A = c_1\ \mathsf{in}\ c_2}\ \text{E-Let}$$

$$\frac{\Gamma;\Delta \vdash e_1 : A \rightsquigarrow c_1 \qquad \Gamma;(d : A) :: \Delta \vdash e_2 : B \rightsquigarrow c_2}{\Gamma;\Delta \vdash \mathsf{impl}\ d : A = e_1\ \mathsf{in}\ e_2 : B \rightsquigarrow \mathsf{impl}\ d : A = c_1\ \mathsf{in}\ c_2}\ \text{E-Impl} \qquad \frac{\Gamma;\Delta \vdash e : A \rightsquigarrow c}{\Gamma;\Delta \vdash (e : A) : A \rightsquigarrow c}\ \text{E-Ascribe}$$

$$\frac{\Gamma \vdash A\ \mathsf{type}}{\Gamma;\Delta \vdash \mathsf{need}[A] : A \rightsquigarrow ?_A}\ \text{E-Need}$$

Expose an ordinary application

$$\frac{T \neq (P \Rightarrow Q)\ \text{for every}\ P, Q}{c : T \triangleright c : T}\ \text{Expose-Stop} \qquad \frac{c\{?_P\} : Q \triangleright c' : T}{c : P \Rightarrow Q \triangleright c' : T}\ \text{Expose-Step}$$

Only ordinary application invokes this relation. In particular, returning an implicit function, referring to it by name, or passing it as an ordinary value does not force it. An implementation initializer is elaborated in the old Δ: bindings are nonrecursive.

4 Typing and elaboration: records, primitives, and method syntax

$$\frac{n \in \mathbb{Z}}{\Gamma; \Delta \vdash n : \mathsf{Int} \rightsquigarrow n}\ \text{E-Int} \qquad \frac{s \text{ a string}}{\Gamma; \Delta \vdash s : \mathsf{String} \rightsquigarrow s}\ \text{E-String} \qquad \frac{b \in \{\mathsf{true}, \mathsf{false}\}}{\Gamma; \Delta \vdash b : \mathsf{Bool} \rightsquigarrow b}\ \text{E-Bool}$$

$$\frac{\Gamma; \Delta \vdash e_j : A_j \rightsquigarrow c_j \quad (j \in J) \qquad \ell_j \text{ distinct}}{\Gamma; \Delta \vdash \{\ell_j = e_j\}_{j \in J} : \{\ell_j : A_j\}_{j \in J} \rightsquigarrow \{\ell_j = c_j\}_{j \in J}}\ \text{E-Record}$$

$$\frac{\Gamma; \Delta \vdash e : \{\ell_j : A_j\}_{j \in J} \rightsquigarrow c \qquad k \in J}{\Gamma; \Delta \vdash e.\ell_k : A_k \rightsquigarrow c.\ell_k}\ \text{E-Project}$$

$$\frac{\Gamma; \Delta \vdash e_1 : \arg(\odot) \rightsquigarrow c_1 \qquad \Gamma; \Delta \vdash e_2 : \arg(\odot) \rightsquigarrow c_2}{\Gamma; \Delta \vdash e_1 \odot e_2 : \mathrm{res}(\odot) \rightsquigarrow c_1 \odot c_2}\ \text{E-BinOp}$$

Receiver-style method selection

For a receiver type A and label ℓ, define the finite set of *types*

$$\mathcal{M}_{\Delta,A,\ell} = \big\{(P, B) \;\big|\; (d : P) \in \Delta,\ P = \{\ell_j : A_j\}_{j \in J},\ \exists k \in J.\ \ell_k = \ell \wedge A_k = A \to B\big\}.$$

Repeated occurrences of the same pair contribute one set member. Let $\mathrm{labels}(A)$ be the field labels when A is a record, and $\varnothing$ otherwise.

$$\frac{\Gamma; \Delta \vdash e : A \rightsquigarrow c \qquad \ell \notin \mathrm{labels}(A) \qquad \mathcal{M}_{\Delta,A,\ell} = \{(P, B)\}}{\Gamma; \Delta \vdash e.\ell : B \rightsquigarrow (?_P).\ell\ c}\ \text{E-Method}$$

Zero candidates reject the selection; two distinct candidate types reject it as ambiguous. This rule determines the request type, not its implementation. Actual record projection has priority; records without the requested field can also use this rule. Explicit qualification $\mathsf{need}[P].\ell(e)$, written `P::member(e)` in source syntax, avoids this receiver-syntax restriction.

The running qualified function

With $\mathsf{ToString}[A] = \{\mathsf{toString} : A \to \mathsf{String}\}$, the source definition

```
def display[A where ToString[A]](x: A): String = x.toString
```

has type and unresolved body

$$\mathsf{display} : \forall A.\mathsf{ToString}[A] \Rightarrow A \to \mathsf{String},$$
$$c_{\mathsf{display}} = \Lambda A.\lambda\{d : \mathsf{ToString}[A]\}.\,\lambda(x : A).\,(?_{\mathsf{ToString}[A]}).\mathsf{toString}\ x.$$

Under $(d : \mathsf{ToString}[A]) :: \Delta$, evidence resolution may select d. After successful resolution:

$$t_{\mathsf{display}} = \Lambda A.\lambda(d : \mathsf{ToString}[A]).\,\lambda(x : A).\,d.\mathsf{toString}\ x, \qquad t_{\mathsf{display}} : \forall A.\mathsf{ToString}[A] \to A \to \mathsf{String}.$$

A concrete call first becomes $c_{\mathsf{display}}[\mathsf{String}]\{?_{\mathsf{ToString}[\mathsf{String}]}\}\ s$. The request is solved in the *caller's* lexical environment, while requests in an ordinary function body are solved in that body's definition environment.

5 Admissible evidence and the synthesizer

Evidence constructions are a restricted subset of the unresolved core:

$$\boxed{p ::= d \mid p[A] \mid p\,p \mid p\{p\}.}$$

The only leaves are references into Δ. New literals, lambdas, projections, ordinary references from Γ, and unresolved requests are not synthesis results. They may occur inside a previously checked first-class value installed in Δ.

$$\frac{d : A \in \Delta}{\Gamma; \Delta \vdash_{\text{ev}} d : A}\ \text{P-Instance} \qquad \frac{\Gamma; \Delta \vdash_{\text{ev}} p : \forall \alpha.A \qquad \Gamma \vdash B\ \mathsf{type}}{\Gamma; \Delta \vdash_{\text{ev}} p[B] : A[B/\alpha]}\ \text{P-Type-App}$$

$$\frac{\Gamma; \Delta \vdash_{\text{ev}} p_1 : A \to B \qquad \Gamma; \Delta \vdash_{\text{ev}} p_2 : A}{\Gamma; \Delta \vdash_{\text{ev}} p_1\,p_2 : B}\ \text{P-App}$$

$$\frac{\Gamma; \Delta \vdash_{\text{ev}} p_1 : A \Rightarrow B \qquad \Gamma; \Delta \vdash_{\text{ev}} p_2 : A}{\Gamma; \Delta \vdash_{\text{ev}} p_1\{p_2\} : B}\ \text{P-Implicit-App}$$

These rules *check* a proposed construction. They do not prescribe a search strategy. A pluggable algorithm supplies a partial judgment

$$\Delta \vdash_{\mathcal{S}} A \rightsquigarrow p.$$

The resolver rechecks $\Gamma; \Delta \vdash_{\text{ev}} p : A$ before lowering. The source arrow kind is checked before erasure, even if two candidate types would erase to the same ordinary function type.

Exact-lookup placeholder

Let $\Delta = [d_0 : A_0, \ldots, d_{m-1} : A_{m-1}]$, nearest binding first.

$$\frac{0 \le j < m \qquad A_j = A \qquad \forall k < j.\ A_k \neq A}{\Delta \vdash_{\text{naive}} A \rightsquigarrow d_j}\ \text{S-Exact}$$

If no j satisfies the premises, the placeholder fails. It does not instantiate a universal type, apply a producer, recurse, or search for a hidden type argument. Failure is not an uninhabitability result.

First-class polymorphic implementations

A generic implementation may have the ordinary core type

$$i : \forall A, B.\mathsf{Repr}[A, B] \Rightarrow \mathsf{ToString}[B] \Rightarrow \mathsf{ToString}[A], \qquad \mathsf{Repr}[A, B] = \{\mathsf{repr} : A \to B\}.$$

Given $r : \mathsf{Repr}[\mathsf{Person}, \mathsf{String}]$ and $s : \mathsf{ToString}[\mathsf{String}]$ in Δ, the construction

$$p = i[\mathsf{Person}][\mathsf{String}]\{r\}\{s\}$$

satisfies $\Gamma; \Delta \vdash_{\text{ev}} p : \mathsf{ToString}[\mathsf{Person}]$. It is permitted as the result of a stronger synthesizer, but the exact-lookup placeholder does not discover it. The source program can construct and install it explicitly:

```
impl person: ToString[Person] = via[Person, String](using personName, using string)
ToString[Person]::toString({ name = "Ada" })
```

There is no special type of implementation values: i, r, s are ordinary values, and impl only changes which context binds them.

6 Resolution and lowering

Type erasure (identity on variables and base types):

$$|A \to B| = |A| \to |B|, \qquad |A \Rightarrow B| = |A| \to |B|,$$
$$|\forall \alpha.A| = \forall \alpha.|A|, \qquad |\{\ell_j : A_j\}_{j\in J}| = \{\ell_j : |A_j|\}_{j\in J}.$$

For evidence, $|d| = d$, $|p[A]| = |p|[|A|]$, and both evidence application forms erase to ordinary application. Ordinary and instance names are embedded in one target namespace; freshness prevents capture.

$$\frac{x : A \in \Gamma}{\Gamma; \Delta \vdash x \Longrightarrow x}\ \text{R-Var} \qquad \frac{d : A \in \Delta}{\Gamma; \Delta \vdash d \Longrightarrow d}\ \text{R-Instance} \qquad \frac{\Delta \vdash_{\mathcal{S}} A \rightsquigarrow p \qquad \Gamma; \Delta \vdash_{\text{ev}} p : A}{\Gamma; \Delta \vdash ?_A \Longrightarrow |p|}\ \text{R-Need}$$

$$\frac{\Gamma, x : A; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash \lambda(x : A).\, c \Longrightarrow \lambda(x : |A|).\, t}\ \text{R-Lambda} \qquad \frac{\Gamma; (d : A) :: \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash \lambda\{d : A\}.\, c \Longrightarrow \lambda(d : |A|).\, t}\ \text{R-Implicit-Lambda}$$

$$\frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash c_1\, c_2 \Longrightarrow t_1\, t_2}\ \text{R-App} \qquad \frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash c_1\{c_2\} \Longrightarrow t_1\, t_2}\ \text{R-Implicit-App}$$

$$\frac{\Gamma, \alpha; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash \Lambda\alpha.c \Longrightarrow \Lambda\alpha.t}\ \text{R-Type-Lambda} \qquad \frac{\Gamma; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash c[A] \Longrightarrow t[|A|]}\ \text{R-Type-App}$$

$$\frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma, x : A; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash \mathsf{let}\ x : A = c_1\ \mathsf{in}\ c_2 \Longrightarrow \mathsf{let}\ x = t_1\ \mathsf{in}\ t_2}\ \text{R-Let}$$

$$\frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; (d : A) :: \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash \mathsf{impl}\ d : A = c_1\ \mathsf{in}\ c_2 \Longrightarrow \mathsf{let}\ d = t_1\ \mathsf{in}\ t_2}\ \text{R-Impl}$$

$$\frac{\Gamma; \Delta \vdash c_j \Longrightarrow t_j \quad (j \in J)}{\Gamma; \Delta \vdash \{\ell_j = c_j\}_{j\in J} \Longrightarrow \{\ell_j = t_j\}_{j\in J}}\ \text{R-Record} \qquad \frac{\Gamma; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash c.\ell \Longrightarrow t.\ell}\ \text{R-Project}$$

$$\frac{}{\Gamma; \Delta \vdash n \Longrightarrow n}\ \text{R-Int} \qquad \frac{}{\Gamma; \Delta \vdash s \Longrightarrow s}\ \text{R-String} \qquad \frac{}{\Gamma; \Delta \vdash b \Longrightarrow b}\ \text{R-Bool}$$

$$\frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash c_1 \odot c_2 \Longrightarrow t_1 \odot t_2}\ \text{R-BinOp}$$

7 Explicit target and independent typing

$$\begin{array}{rcl} T, U & ::= & \alpha \mid \mathsf{Int} \mid \mathsf{Bool} \mid \mathsf{String} \mid T \to U \mid \forall\alpha.T \mid \{\ell_j : T_j\}_{j\in J} \\ t & ::= & z \mid n \mid b \mid s \mid \lambda(z:T).\,t \mid t\,t \mid \Lambda\alpha.t \mid t[T] \\ & & \mid \mathsf{let}\ z = t\ \mathsf{in}\ t \mid \{\ell_j = t_j\}_{j\in J} \mid t.\ell \mid t_1 \odot t_2 \\ H & ::= & \cdot \mid H, \alpha \mid H, z : T \end{array}$$

Target type formation is Section 1 with only the ordinary arrow. All annotations and type arguments must be well formed in H. All record labels are distinct.

$$\frac{z : T \in H}{H \vdash z : T}\ \text{T-Var} \qquad \frac{H, z : T \vdash t : U}{H \vdash \lambda(z:T).\,t : T \to U}\ \text{T-Lambda} \qquad \frac{H \vdash t_1 : T \to U \qquad H \vdash t_2 : T}{H \vdash t_1\,t_2 : U}\ \text{T-App}$$

$$\frac{H, \alpha \vdash t : T \qquad \alpha \notin \mathrm{FV}(H)}{H \vdash \Lambda\alpha.t : \forall\alpha.T}\ \text{T-Type-Lambda} \qquad \frac{H \vdash t : \forall\alpha.T \qquad H \vdash U\ \mathsf{type}}{H \vdash t[U] : T[U/\alpha]}\ \text{T-Type-App}$$

$$\frac{H \vdash t_1 : T \qquad H, z : T \vdash t_2 : U}{H \vdash \mathsf{let}\ z = t_1\ \mathsf{in}\ t_2 : U}\ \text{T-Let} \qquad \frac{H \vdash t_j : T_j \quad (j \in J)}{H \vdash \{\ell_j = t_j\}_{j\in J} : \{\ell_j : T_j\}_{j\in J}}\ \text{T-Record}$$

$$\frac{H \vdash t : \{\ell_j : T_j\}_{j\in J} \qquad k \in J}{H \vdash t.\ell_k : T_k}\ \text{T-Project} \qquad \frac{n \in \mathbb{Z}}{H \vdash n : \mathsf{Int}}\ \text{T-Int} \qquad \frac{s\ \text{a string}}{H \vdash s : \mathsf{String}}\ \text{T-String}$$

$$\frac{b \in \{\mathsf{true}, \mathsf{false}\}}{H \vdash b : \mathsf{Bool}}\ \text{T-Bool} \qquad \frac{H \vdash t_1 : \arg(\odot) \qquad H \vdash t_2 : \arg(\odot)}{H \vdash t_1 \odot t_2 : \mathrm{res}(\odot)}\ \text{T-BinOp}$$

Phase invariant and index representation

Let $H = |\Gamma| \cup |\Delta|$ merge the two term environments using the same globally fresh names. The phase invariant is

$$\boxed{\Gamma; \Delta \vdash e : A \rightsquigarrow c \quad \wedge \quad \Gamma; \Delta \vdash c \Longrightarrow t \quad \Longrightarrow \quad H \vdash t : |A|.}$$

The reference implementation independently checks the final judgment; no mechanized metatheory is asserted here. The synthesizer is not a trusted source of typing derivations.

In the implementation, ordinary and instance references have independent de Bruijn indices. The resolver maintains one embedding for each into the current target environment. Every new term binder increments all previously embedded target indices. An ordinary binder then extends only the Γ embedding; an implementation binder extends only the Δ embedding. A type binder weakens type indices in both environments and in embedded evidence. Type substitution is capture avoiding.

8 Call-by-value evaluation

$$\begin{array}{lcl} v & ::= & n \mid b \mid s \mid \{\ell_j = v_j\}_{j\in J} \mid \langle \lambda z.t, \rho\rangle \mid \langle \Lambda\alpha.t, \rho\rangle, \\ \rho & ::= & \cdot \mid \rho[z \mapsto v]. \end{array}$$

Evaluation ignores type annotations and type arguments. Type abstraction creates a delayed closure; type application forces it. Records are evaluated in canonical label order.

$$\frac{\rho(z) = v}{\rho \vdash z \Downarrow v}\ \text{V-Var} \qquad \frac{}{\rho \vdash n \Downarrow n}\ \text{V-Int} \qquad \frac{}{\rho \vdash s \Downarrow s}\ \text{V-String} \qquad \frac{}{\rho \vdash b \Downarrow b}\ \text{V-Bool}$$

$$\frac{}{\rho \vdash \lambda(z : T).\, t \Downarrow \langle \lambda z.t, \rho\rangle}\ \text{V-Lambda} \qquad \frac{}{\rho \vdash \Lambda\alpha.t \Downarrow \langle \Lambda\alpha.t, \rho\rangle}\ \text{V-Type-Lambda}$$

$$\frac{\rho \vdash t_1 \Downarrow \langle \lambda z.t, \rho_0\rangle \qquad \rho \vdash t_2 \Downarrow v_2 \qquad \rho_0[z \mapsto v_2] \vdash t \Downarrow v}{\rho \vdash t_1\, t_2 \Downarrow v}\ \text{V-App}$$

$$\frac{\rho \vdash t_0 \Downarrow \langle \Lambda\alpha.t, \rho_0\rangle \qquad \rho_0 \vdash t \Downarrow v}{\rho \vdash t_0[T] \Downarrow v}\ \text{V-Type-App} \qquad \frac{\rho \vdash t_1 \Downarrow v_1 \qquad \rho[z \mapsto v_1] \vdash t_2 \Downarrow v}{\rho \vdash \mathsf{let}\ z = t_1\ \mathsf{in}\ t_2 \Downarrow v}\ \text{V-Let}$$

$$\frac{\rho \vdash t_j \Downarrow v_j \quad (j \in J)}{\rho \vdash \{\ell_j = t_j\}_{j\in J} \Downarrow \{\ell_j = v_j\}_{j\in J}}\ \text{V-Record} \qquad \frac{\rho \vdash t \Downarrow \{\ell_j = v_j\}_{j\in J} \qquad k \in J}{\rho \vdash t.\ell_k \Downarrow v_k}\ \text{V-Project}$$

$$\frac{\rho \vdash t_1 \Downarrow v_1 \qquad \rho \vdash t_2 \Downarrow v_2 \qquad \delta_\odot(v_1, v_2) = v}{\rho \vdash t_1 \odot t_2 \Downarrow v}\ \text{V-BinOp}$$

Closed-program execution

$$\frac{\cdot;[\,] \vdash e : A \rightsquigarrow c \qquad \cdot;[\,] \vdash c \Longrightarrow t \qquad \cdot \vdash t : |A| \qquad \cdot \vdash t \Downarrow v}{e \Downarrow_{\mathcal{S}} v}\ \text{Execute}$$

There is no runtime evidence search. Resolving an implementation binding produces an ordinary target let, so its value can close over ordinary runtime variables. The compiled value remains first-class; registration affects only lexical evidence scope.